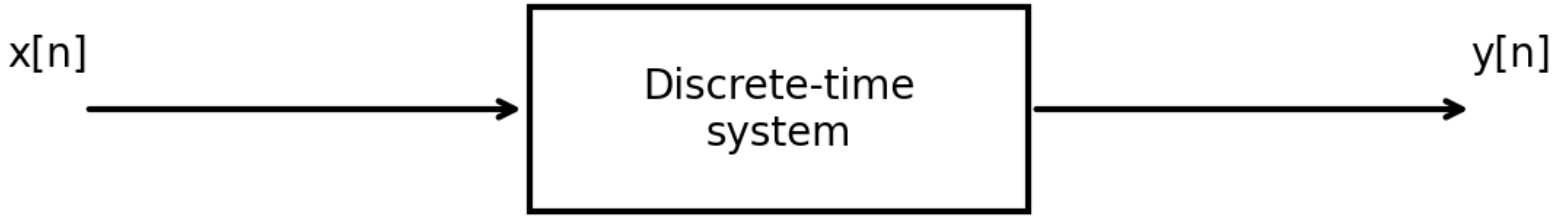
Topic 5

FIR Filters

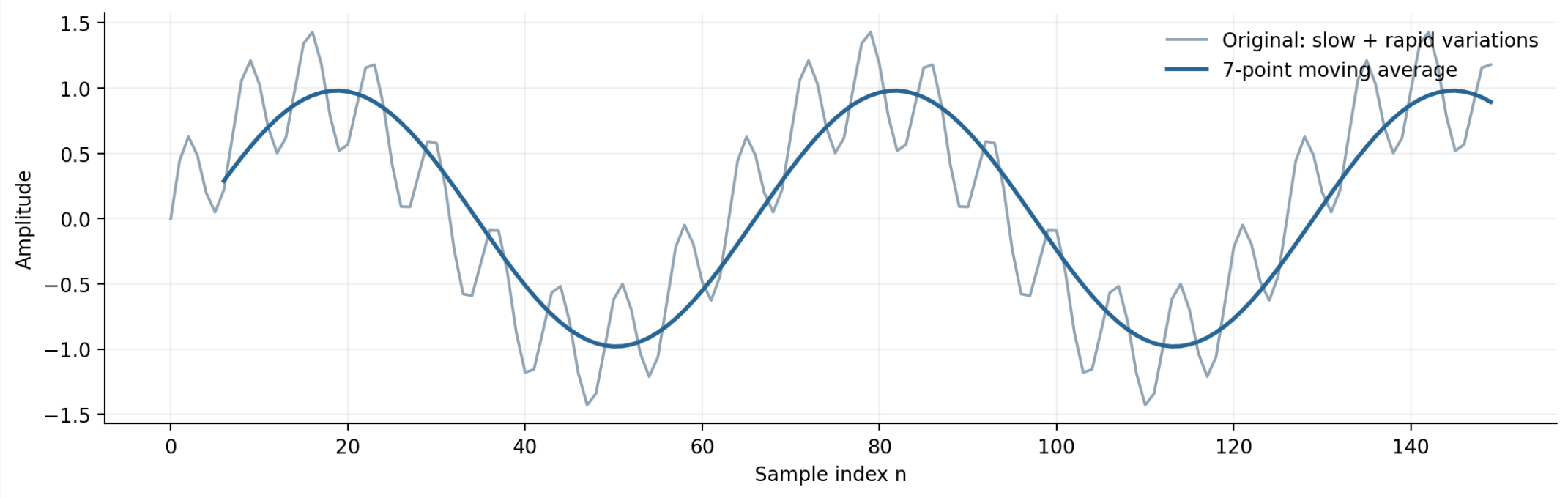
Discrete-time systems, impulse response, convolution, and LTI systems



Key idea. An FIR filter computes each output sample from a finite set of present and past input samples.

Why filter a signal?

A filter can preserve slow variations while reducing fast fluctuations.



Discrete-time systems

A system transforms an input sequence into an output sequence

$$y[n] = T\{x[n]\}$$

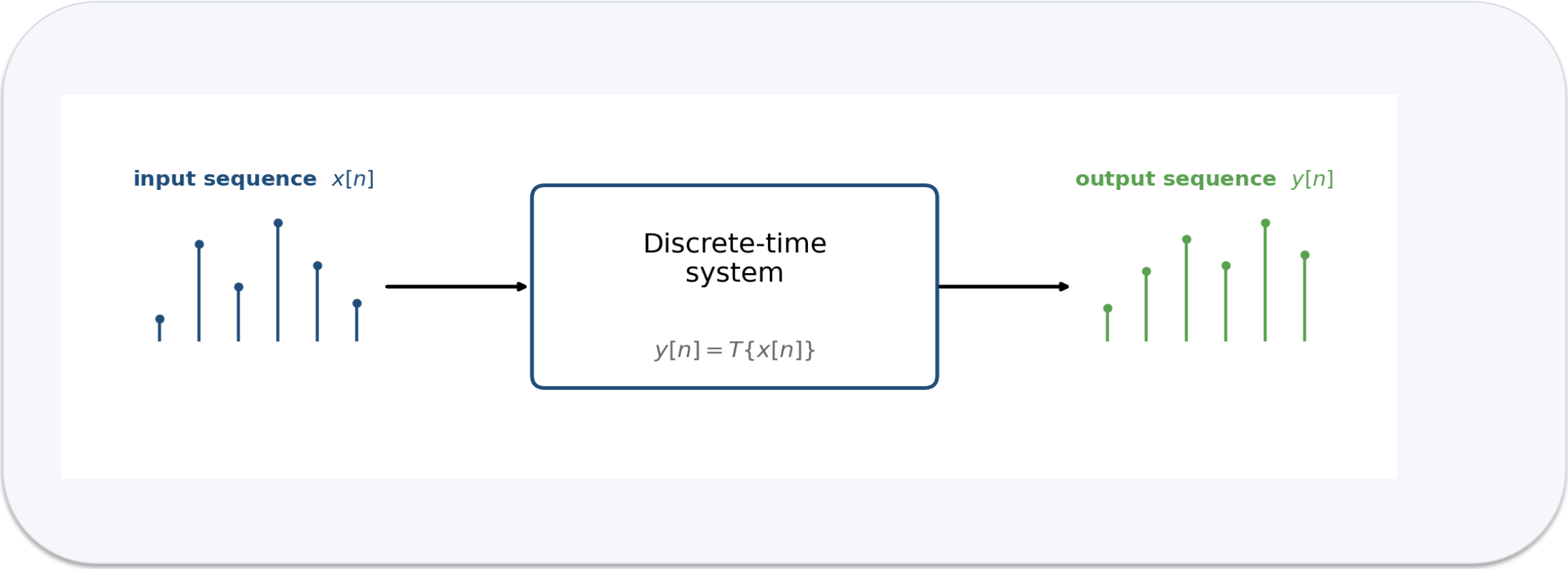
Example: $y[n] = a \cdot x[n]$

- A discrete-time system is a computational process acting on the input x[n].
- The output y[n] can scale, smooth, delay, combine, or otherwise transform the sequence.
- We study systems because filters are the building blocks of digital signal processing.

Key idea. A filter is best understood as an input-output rule.

Discrete-time systems — block diagram

Basic input-output view of a discrete-time system



A system transforms the input sequence x[n] into the output sequence y[n].

Not every system is a linear filter

A discrete-time system is a broad idea. FIR filters are a particular class.

Pointwise scaling

$$y[n] = 2x[n]$$

Linear; memoryless

Squaring

$$y[n] = x[n]^2$$

Nonlinear; memoryless

3-point average

$$y[n] = (x[n]+x[n-1]+x[n-2])/3$$

Linear; has memory

A system may transform samples independently or depend on neighboring samples; linearity must be checked, not assumed.

Moving-average filter

A simple FIR filter that smooths rapid fluctuations

3-point causal average: $y[n] = \frac{1}{3}(x[n] + x[n-1] + x[n-2])$

$$L - point\ average: \quad y[n] = (1/L) \sum_{k=0}^{L-1} x[n-k]$$

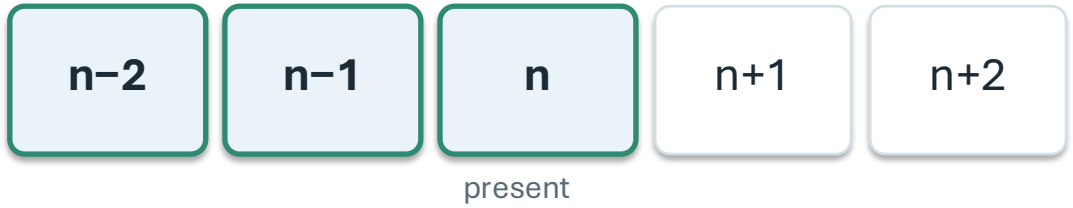
- Each output sample is the average of a short window of neighboring input samples.
- Longer windows produce stronger smoothing but also more delay and less temporal detail.
- A centred moving average is usually noncausal because it needs future samples.

Key idea. A moving average smooths by spreading each sample across a short time window.

Causal or centered moving average?

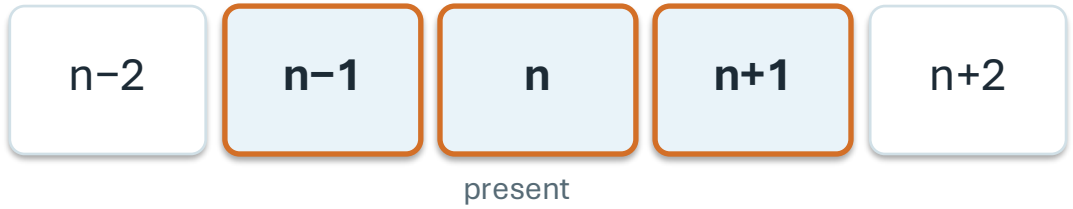
The window position determines whether future input samples are required.

Causal: available in real time



$$y[n] = (x[n]+x[n-1]+x[n-2])/3$$

Centered: requires a future value



$$y[n] = (x[n-1]+x[n]+x[n+1])/3$$

Causal FIRs use x[n] and past samples only; the centered average is not causal in this alignment.

One 3-point average, three time alignments

The coefficients can be identical even when the selected samples have different indices.

Forward-looking $y[n] = (x[n]+x[n+1]+x[n+2])/3$

Uses two future values

Centered $y[n] = (x[n-1]+x[n]+x[n+1])/3$

Uses one future value

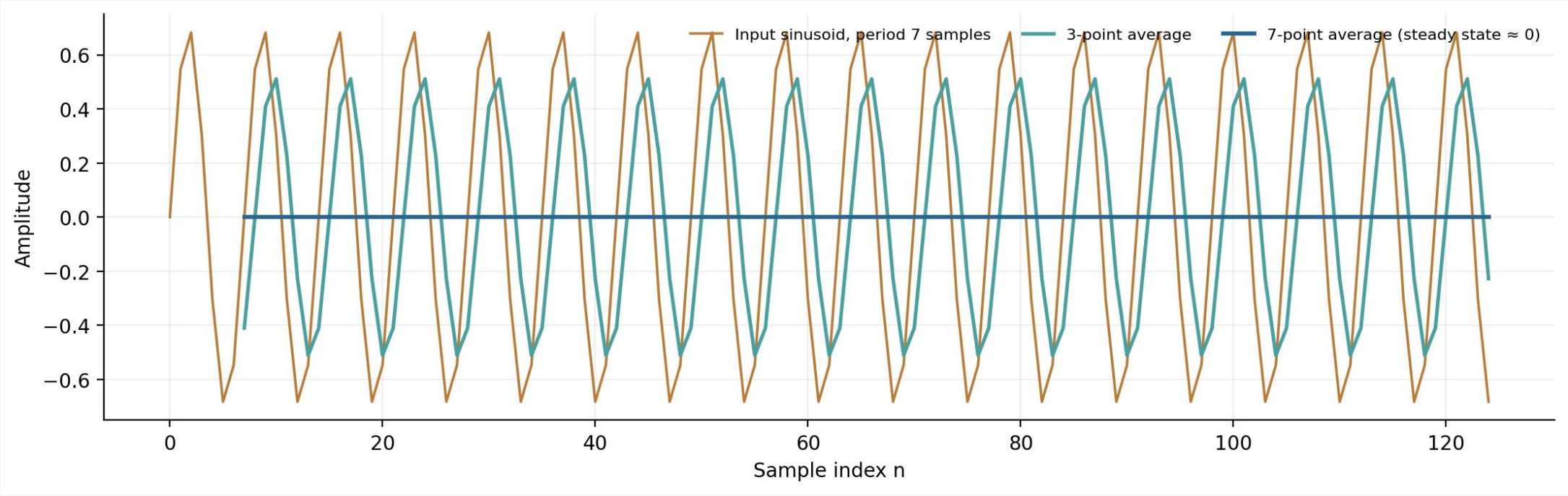
Causal $y[n] = (x[n]+x[n-1]+x[n-2])/3$

Uses present and past

For the input $x[0..4]=\{2,4,6,4,2\}$, all three contain the same values, but their output time indices differ.

Why can a longer average suppress a sinusoid?

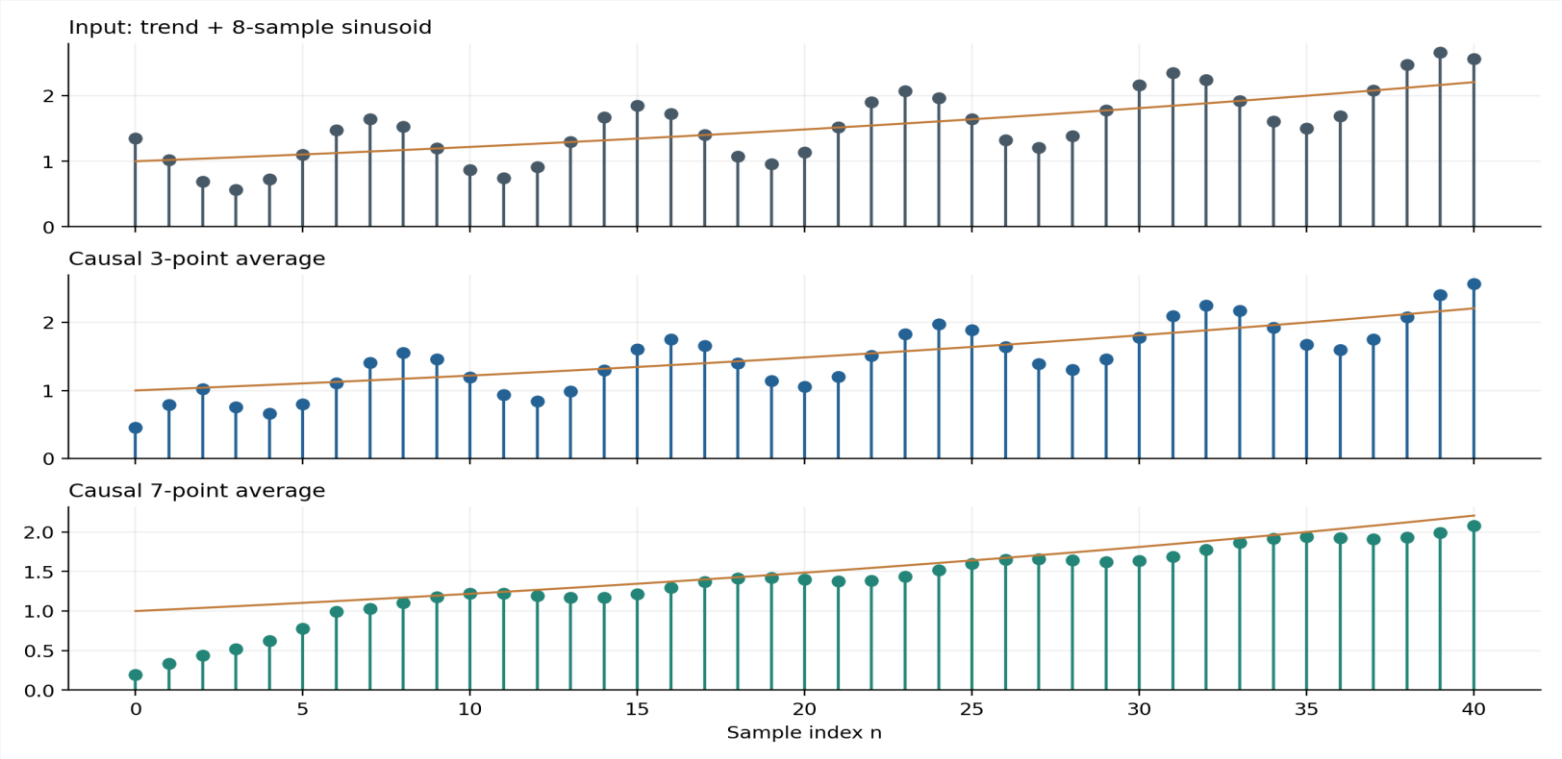
An average can cancel positive and negative samples of a fast oscillation.



This period-7 illustration shows exact cancellation with 7 taps. The textbook example in Section 5.3 uses a period-8 disturbance.

The textbook moving-average example

A slowly increasing trend plus a period-8 sinusoidal disturbance.



The 7-point average strongly reduces rapid fluctuation, but its onset/offset effects and delay are larger.

Signal Processing First, Chapter 5, Fig. 5.6 (recomputed)

Example 1 — moving average

Causal 3-point average: calculate the complete output from the input samples.

$y[n] = (x[n] + x[n-1] + x[n-2]) / 3$

Input: x[0..4] = [2, 4, 6, 4, 2]; zero outside.

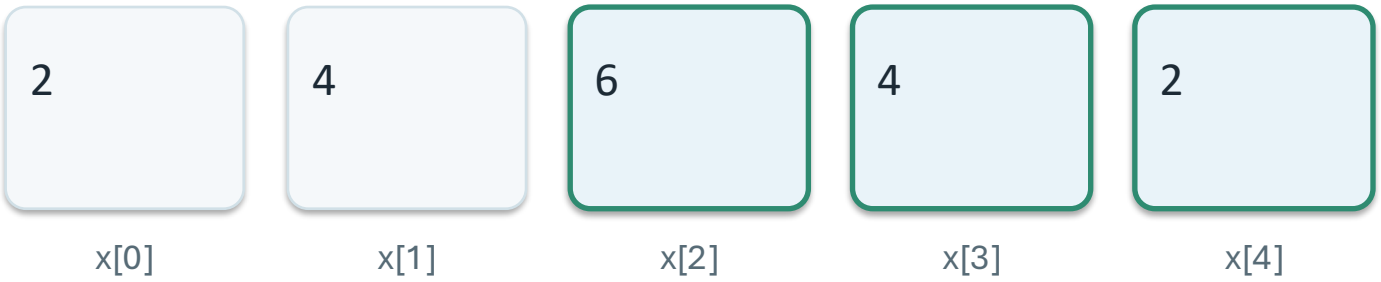
n	0	1	2	3	4	5	6
x[n]	2	4	6	4	2	0	0
y[n]	⅔	**2**	**4**	**14/3**	**4**	**2**	⅔

Example: $y[3] = (x[3]+x[2]+x[1])/3 = (4+6+4)/3 = 14/3$

The output has 5 + 3 − 1 = 7 samples; the same filter coefficients are reused at every position.

Calculate one output sample with the sliding window

Causal 3-point average: x = {2, 4, 6, 4, 2}, zero outside 0 ≤ n ≤ 4.



$$y[4] = (x[4] + x[3] + x[2]) / 3$$

$$= (2 + 4 + 6) / 3 = 4$$

The same 3 coefficients are reused at every output position; only the input window shifts.

General FIR filter

Difference equation, coefficient list, and causality

$$y[n] = \sum_{k=0}^{M} b[k] \cdot x[n-k]$$

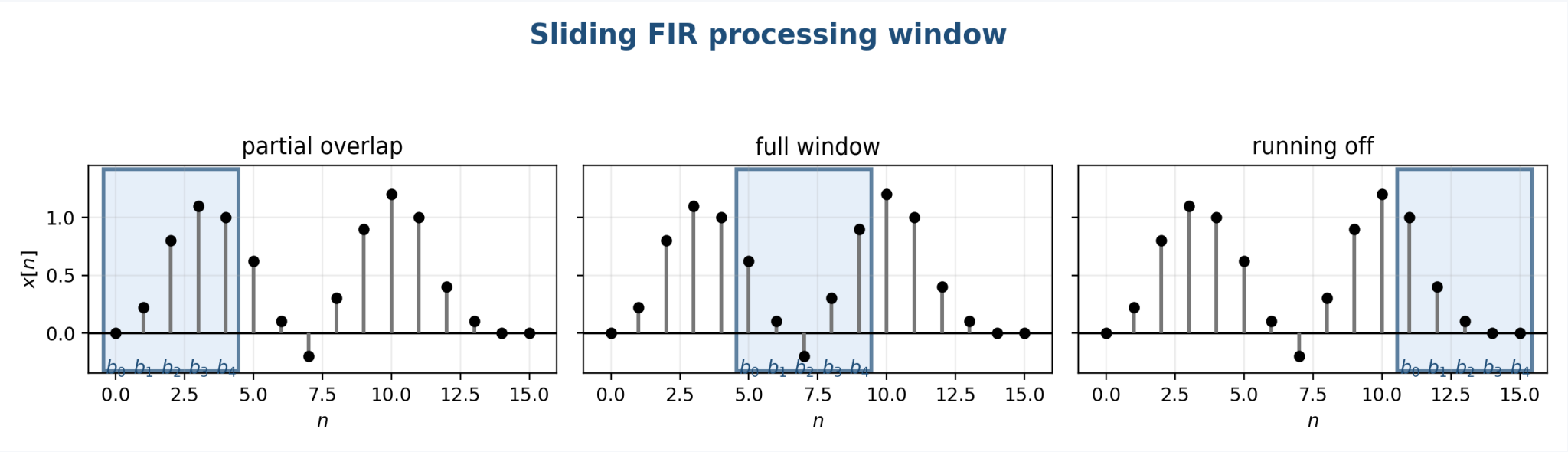
$$\text{Order: M} \qquad \text{Length: L} = \text{M} + 1$$

- The coefficients b[0], …, b[M] specify how the delayed input samples are weighted.
- The filter is causal because it uses only the present sample and past samples x[n−k].
- The coefficient list, difference equation, and impulse response are equivalent descriptions.

Key idea. Any causal FIR filter is a weighted sum of delayed copies of the input.

General FIR filter — window interpretation

The FIR output is a weighted sum over a finite window



A causal FIR filter combines M+1 consecutive samples as the processing window slides along the data.

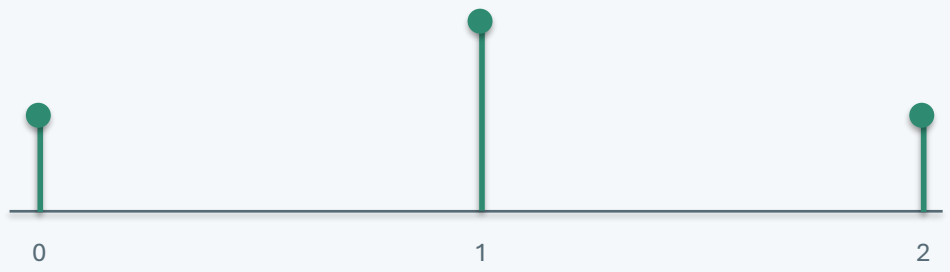
Three descriptions of the same FIR filter

Coefficients, difference equation, and impulse response carry the same information.

$$b = [¼, ½, ¼]$$

$$y[n] = ¼x[n] + ½x[n-1] + ¼x[n-2]$$

Impulse response h[n]



Length L = 3 taps
Order M = 2 delays

For a causal FIR, h[k] = b[k] over its finite duration. The “impulse response” is the coefficient sequence.

Example 2 — FIR coefficients and order

Reading the difference equation directly from the coefficient list

Problem

Let the FIR coefficients be b[0]=3, b[1]=−1, b[2]=2, and b[3]=1. Write the difference equation and identify the order and length.

Solution

1. Insert the coefficients into $y[n] = \Sigma\, b[k]x[n-k]$.
2. This gives $y[n] = 3x[n] - x[n-1] + 2x[n-2] + x[n-3]$.
3. There are four coefficients, so the filter length is $L = 4$.
4. The highest delay is 3, so the filter order is $M = 3$.

Final answer: $y[n] = 3x[n] - x[n-1] + 2x[n-2] + x[n-3]$, with order $M = 3$ and length $L = 4$.

Key idea. For FIR filters, the order is the largest delay and the length is the number of taps.

Unit impulse and impulse response

The building blocks of FIR and LTI analysis

$\delta[n] = 1$ at $n=0$, and 0 otherwise

$h[n]$ = output when $x[n] = \delta[n]$

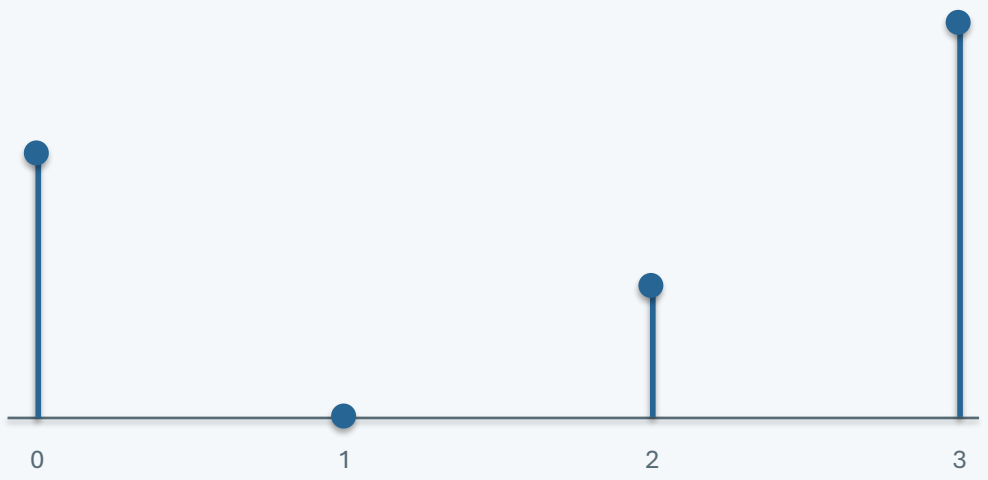
- A shifted impulse $\delta[n-n_0]$ is nonzero only at $n = n_0$.
- Any sequence can be written as $x[n] = \Sigma\, x[k]\delta[n-k]$, a sum of scaled, shifted impulses.
- For a causal FIR filter, $h[n]$ is exactly the coefficient sequence placed at the appropriate delays.
- A unit delay is the simplest FIR system: $y[n] = x[n-1]$.

Key idea. Knowing the impulse response means knowing the filter.

Any discrete-time signal is a sum of shifted impulses

The coefficients of each impulse are simply the samples of the signal.

Example signal: x[n]



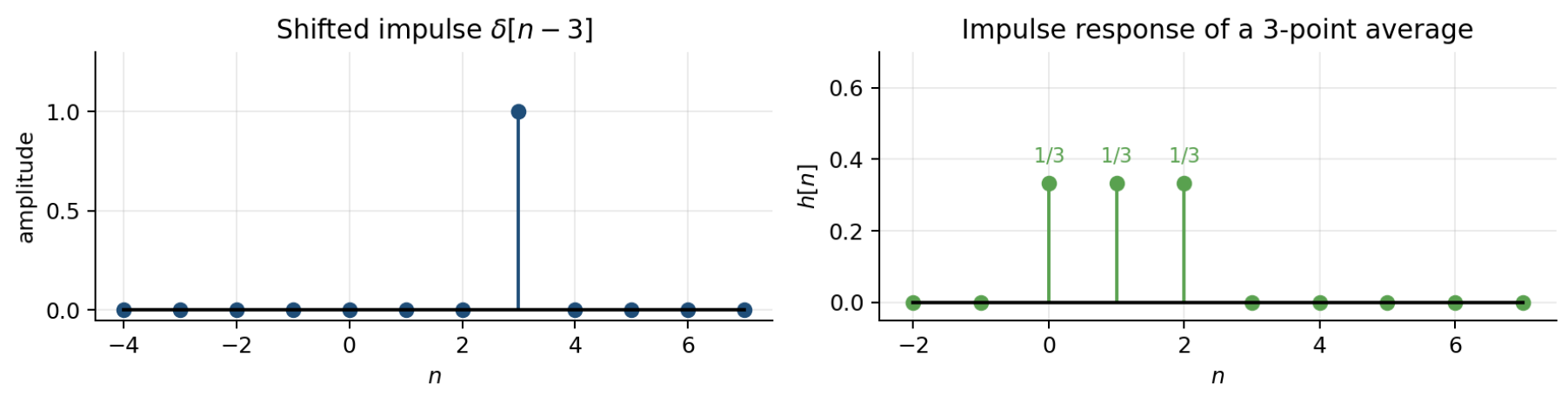
$$x[n] = 2\delta[n] + \delta[n-2] + 3\delta[n-3]$$

One impulse at n = 0, another at n = 2, and another at n = 3.

General rule: $x[n] = \Sigma_k\, x[k]\, \delta[n-k]$. This is the key step leading to the convolution sum.

Unit impulse and impulse response — visual examples

A shifted impulse and the impulse response of a 3-point average

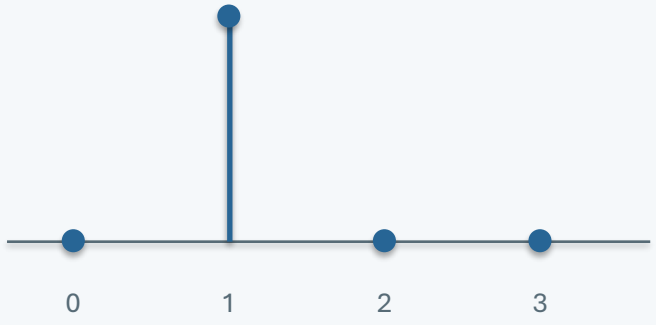


The unit impulse is the building block of signals, and the impulse response fully characterizes an LTI FIR filter.

Compare three impulse responses

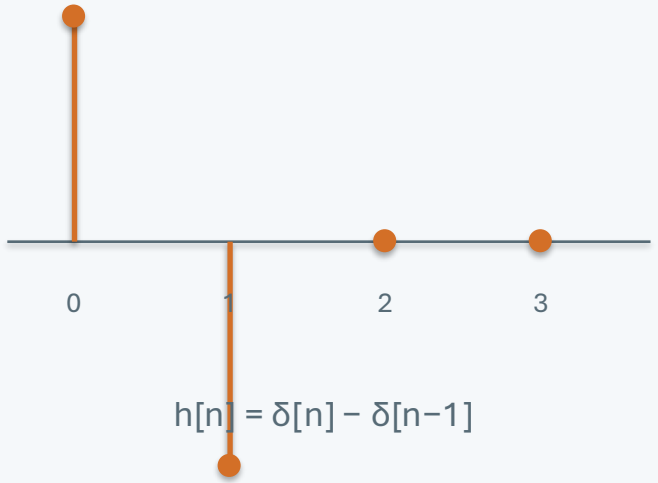
The response to δ[n] reveals what the FIR filter does.

Unit delay



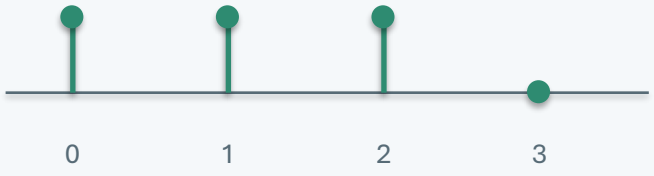
$h[n] = \delta[n-1]$

First difference



$h[n] = \delta[n] - \delta[n-1]$

3-point average



$h[n] = (\delta[n]+\delta[n-1]+\delta[n-2])/3$

Impulse responses can have positive, negative, or zero values. They need not resemble the input signal.

Example 3 — impulse response

From the difference equation to h[n] and shifted-impulse outputs

Problem

A causal FIR system is $y[n] = 2x[n] - x[n-1] + (1/2)x[n-3]$. Find $h[n]$, and then the output for input $\delta[n-4]$.

Solution

1. Set $x[n]=\delta[n]$; the impulse response is $h[n] = 2\delta[n] - \delta[n-1] + (1/2)\delta[n-3]$.
2. Thus the nonzero samples are $h[0]=2$, $h[1]=-1$, and $h[3]=1/2$.
3. A delayed impulse at the input produces the same delay at the output.
4. Therefore the response to $\delta[n-4]$ is $h[n-4] = 2\delta[n-4] - \delta[n-5] + (1/2)\delta[n-7]$.

Final answer: $h[n] = 2\delta[n] - \delta[n-1] + (1/2)\delta[n-3]$, and for input $\delta[n-4]$ the output is $2\delta[n-4] - \delta[n-5] + (1/2)\delta[n-7]$.

Convolution and FIR filters

A compact way to compute the output of a filter

$$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n-k]$$

For finite sequences: $\text{length}(y) = \text{length}(x) + \text{length}(h) - 1$

- Convolution adds scaled and shifted copies of one sequence, weighted by the other.
- For FIR filters, convolution and the difference equation produce exactly the same output.
- Convolution is commutative: x*h = h*x. It is also associative.

Key idea. Convolution is the natural algebra of LTI systems and FIR filtering.

Two equivalent ways to understand convolution

Both compute the same y[n] but suggest different mental pictures.

Sliding-window view

$$y[n] = \Sigma_k\, h[k] \cdot x[n-k]$$

At each n, multiply the input samples in the window by the filter coefficients, then add.

Shifted-response view

$$y[n] = \Sigma_k\, x[k] \cdot h[n-k]$$

Each input sample generates a scaled, shifted copy of h[n]. Add all the copies.

They are the same expression after changing the summation index; convolution is commutative.

Example 4 — numerical convolution

Computing the FIR output sample by sample

Problem

Let $x[n] = \{2,4,6,4,2\}$ and $h[n] = \{3,-1,2,1\}$, both starting at $n = 0$. Compute $y[n] = x[n] * h[n]$.

Solution

1. The output length is $5 + 4 - 1 = 8$ samples.
2. Start at the edges: $y[0]=2\cdot3=6$ and $y[1]=4\cdot3 + 2\cdot(-1)=10$.
3. Continue the overlap-add calculation or use the shifted-copy view of convolution.
4. The full result is $y[n] = \{6,10,18,16,18,12,8,2\}$.

Final answer: $y[n] = \{6,10,18,16,18,12,8,2\}$. Reversing the order to h*x gives the same result.

Convolution as shifted and scaled impulse responses

A compact example: x = [1, 2, 1], h = [1, −1].

	n = 0	n = 1	n = 2	n = 3
x[0]h[n]	1	-1	0	0
x[1]h[n−1]	0	2	-2	0
x[2]h[n−2]	0	0	1	-1
sum y[n]	1	1	-1	-1

Here the filter has two taps. A 3-sample input produces 3 + 2 − 1 = 4 output samples.

Convolution — numerical interpretation

Summing scaled and shifted copies of the input

Each row is one shifted, scaled copy of the input; the last row is their sum.

n	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
$h[0]x[n-0]$	6	12	18	12	6	0	0	0
$h[1]x[n-1]$	0	-2	-4	-6	-4	-2	0	0
$h[2]x[n-2]$	0	0	4	8	12	8	4	0
$h[3]x[n-3]$	0	0	0	2	4	6	4	2
$y[n]$	6	10	18	16	18	12	8	2

This table shows how each shifted term contributes to the final convolution output y[n].

Convolution: check one output position

At n = 2, only the samples that overlap the filter matter.

x = [2, 4, 6, 4, 2] h = [3, −1, 2, 1]

$$y[2] = h[0]x[2] + h[1]x[1] + h[2]x[0]$$

3 × 6

(−1) × 4

2 × 2

18 − 4 + 4 = 18

In a convolution sum, the sample indices add up to n: h[k] multiplies x[n−k].

FIR-filter implementation

Direct-form realization uses delays, multipliers, and adders

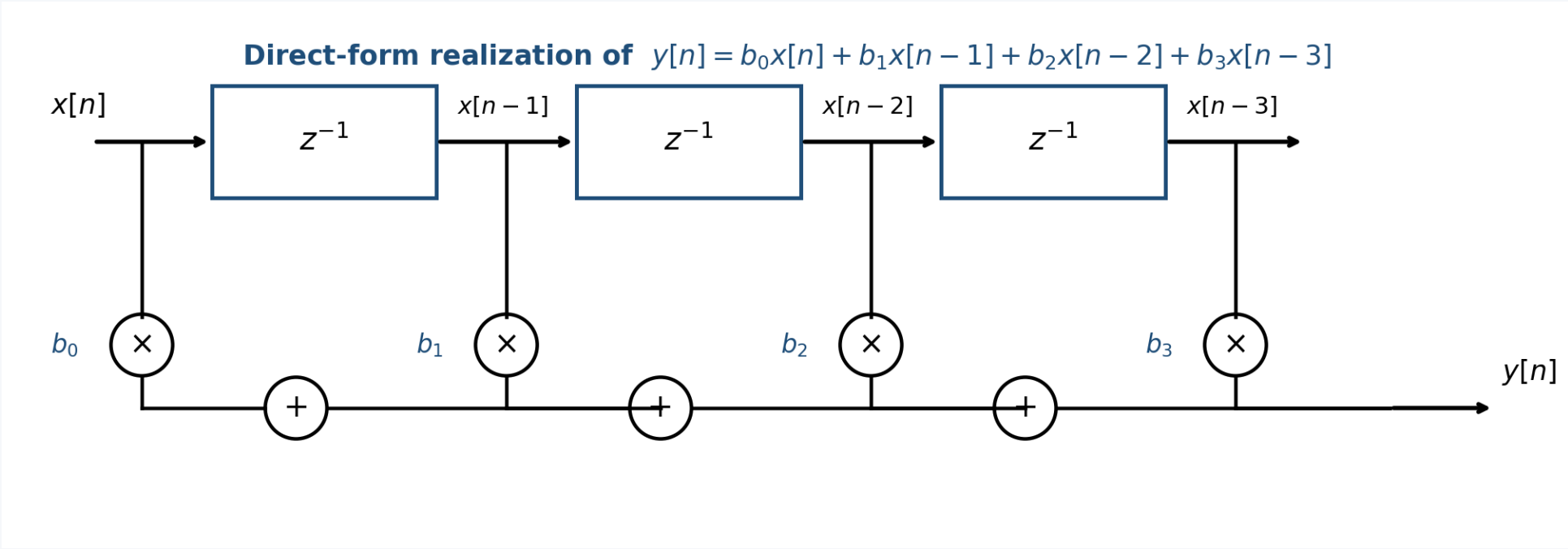
Direct form implements $y[n] = b[0]x[n] + b[1]x[n-1] + \cdots + b[M]x[n-M]$

- The input passes through a chain of unit delays to create $x[n-1]$, $x[n-2]$, ..., $x[n-M]$.
- Each branch is multiplied by its coefficient $b[k]$, and the branches are summed.
- A length-L FIR filter needs $M = L-1$ delay elements, L multipliers, and $L-1$ additions.
- Alternative block diagrams are equivalent if they implement the same input-output rule.

Key idea. Implementation details may change, but the realized FIR system stays the same.

FIR-filter implementation — direct form

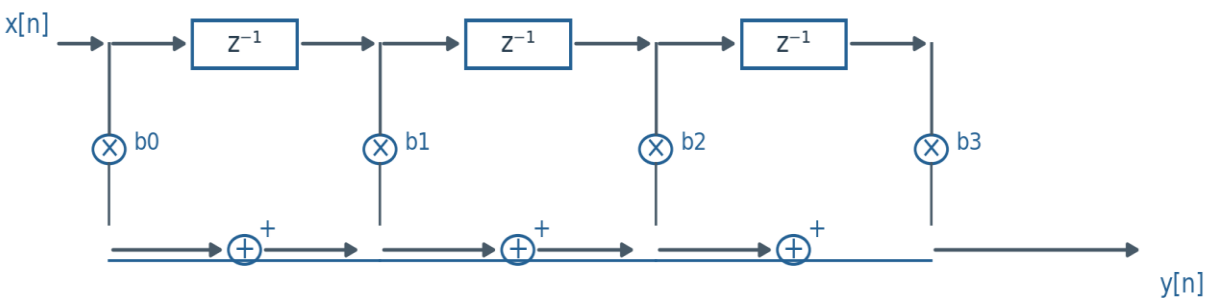
Realization with unit delays, multipliers, and adders



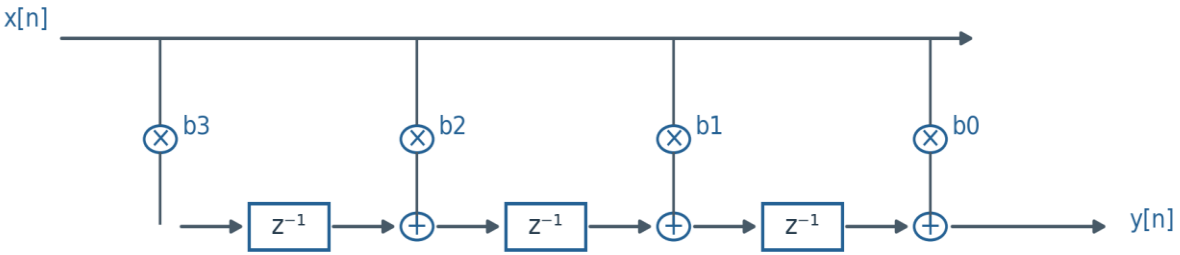
A direct-form FIR implementation creates delayed versions of x[n], weights them by b[k], and sums the branches.

Direct-form and transposed-form FIR filters

The internal signals differ, but the input–output system is the same.



Four-tap FIR: $y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] + b_3x[n-3]$



Same four coefficients and exactly the same external input–output relationship

Only three hardware building blocks are needed

A direct-form FIR combines delay, multiplication, and addition.

Delay

z^{-1}

output is x[n−1]

Multiplier

× b[k]

scales one delayed sample

Adder

+

sums the tap contributions

For an L-tap direct-form FIR: L multipliers, L−1 delays and L−1 additions (without exploiting coefficient structure).

Linearity and time invariance

Why FIR systems are easy to analyze

Linearity: $T\{\alpha x_1[n] + \beta x_2[n]\} = \alpha T\{x_1[n]\} + \beta T\{x_2[n]\}$

Time invariance: if $y[n]=T\{x[n]\}$, then $T\{x[n-n_0]\} = y[n-n_0]$

- Linearity combines homogeneity and additivity: scale inputs and outputs scale the same way.
- Time invariance means a time shift at the input produces the same shift at the output.
- The general FIR system is LTI because its delays and coefficients are fixed.

Key idea. LTI properties make convolution possible and give a complete system description through h[n].

Test linearity with an easy counterexample

A system is linear only if it respects both scaling and superposition.

$$T\{x[n]\} = x[n]^2$$

Choose two inputs: $x_1[n] = 1$ and $x_2[n] = 1$.

$$T\{x_1 + x_2\} = (1 + 1)^2 = 4$$

$$T\{x_1\} + T\{x_2\} = 1^2 + 1^2 = 2$$

$$4 \neq 2 \quad \Rightarrow \quad \text{squaring is not linear}$$

Contrast: any fixed-coefficient FIR sum is linear; delays, scaling and addition all preserve linearity.

Time invariance: shift the input, shift the output

A time-varying gain is a useful example of what fails.

$$T\{x[n]\} = n \cdot x[n]$$

Input at n = 0

$x[n] = \delta[n] \rightarrow y[n] = n\delta[n] = 0$

Shift input by 1

$x_1[n] = \delta[n-1] \rightarrow y_1[n] = n\delta[n-1] = \delta[n-1]$

The shifted first output is still zero, but the second output is $\delta[n-1]$: therefore T is not time invariant.

Linearity and time invariance are different tests

A system may satisfy one property without satisfying the other.

System	Linear?	Time invariant?
$y[n] = x[n]^2$	**No**	**Yes**
$y[n] = n \cdot x[n]$	**Yes**	**No**
$y[n] = x[-n]$	**Yes**	**No**
$y[n] = x[n] - x[n-1]$	**Yes**	**Yes**

All fixed-coefficient FIR filters are LTI; the examples above show why both properties must be checked.

Convolution and LTI systems

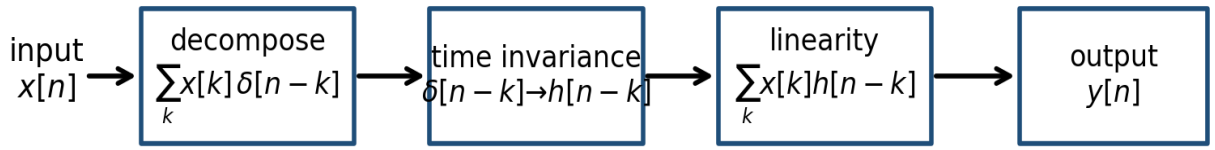
The impulse response completely characterizes an LTI system

$$x[n] = \sum_k x[k] \cdot \delta[n-k]$$

$$y[n] = \sum_k x[k] \cdot h[n-k]$$

- Decompose the input into scaled, shifted impulses.
- By time invariance, δ[n−k] produces h[n−k].
- By linearity, the total output is the sum of those responses.
- Therefore every LTI system is represented by a convolution with h[n].

LTI viewpoint behind convolution



Every scaled, shifted impulse produces a scaled, shifted copy of the impulse response.

Key idea. Impulse decomposition + LTI properties = convolution sum.

Why LTI systems lead to convolution

Impulse decomposition + time invariance + linearity = one formula.

Start	$x[n] = \Sigma_k x[k]\delta[n-k]$	Any signal is a sum of impulses
Time invariance	$\delta[n-k] \rightarrow h[n-k]$	Shifted input impulse → shifted response
Linearity	$y[n] = \Sigma_k x[k]h[n-k]$	Scale and add every impulse response

Every LTI system is completely characterized by its impulse response h[n], whether FIR or IIR.

All fixed-coefficient FIR systems are LTI

But the convolution sum applies to a broader class of systems than finite impulse responses.

General LTI

$y[n] = \Sigma_k x[k]h[n-k]$ (sum over all integer k)

Causal FIR

$h[k] = 0$ outside $0 \leq k \leq M$

Key distinction

Finite h[n] → FIR. Some other LTI systems have impulse responses that extend indefinitely.

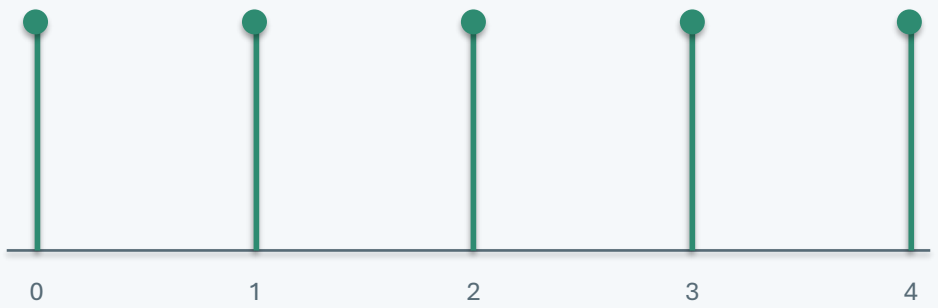
The impulse response h[n] completely characterizes any LTI system, not only a causal FIR filter.

In-class example: a first-difference filter

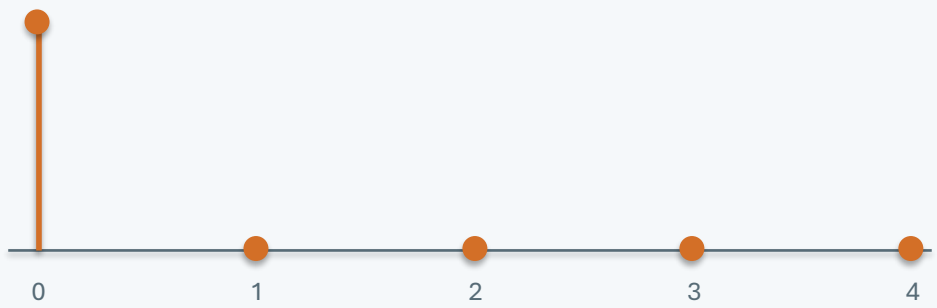
What is the response when a unit step enters the system?

$$y[n] = x[n] - x[n-1]$$

Unit step u[n] (n ≥ 0)



Output y[n] = δ[n]



The first difference removes the constant region and detects the transition. Its impulse response is [1, −1].

Verify convolution with Python

One line of code reproduces the numerical example in the slides.

```
import numpy as np
x = np.array([2, 4, 6, 4, 2])
h = np.array([3, -1, 2, 1])
y = np.convolve(x, h)
```

```
y = [6, 10, 18, 16,
   18, 12, 8, 2]
```

Cascaded LTI systems

Two FIR filters in sequence are equivalent to a single FIR filter

$h_eq[n] = h_1[n] * h_2[n]$

Because convolution is commutative: $h_1 * h_2 = h_2 * h_1$

- Placing one LTI system after another produces a new LTI system.
- The equivalent impulse response is the convolution of the individual impulse responses.
- The order of the cascade does not change the overall result.

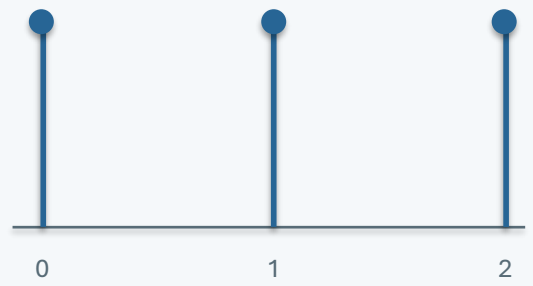
Key idea. Cascades let us build complicated filters from simpler ones.

Cascading two averages creates a smoother filter

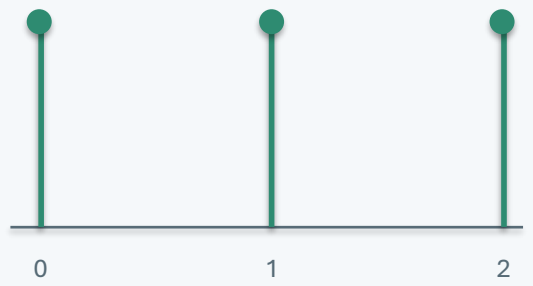
Equivalent impulse response = convolution of the two impulse responses.

$$h_1 = [⅓, ⅓, ⅓] \quad h_2 = [⅓, ⅓, ⅓]$$

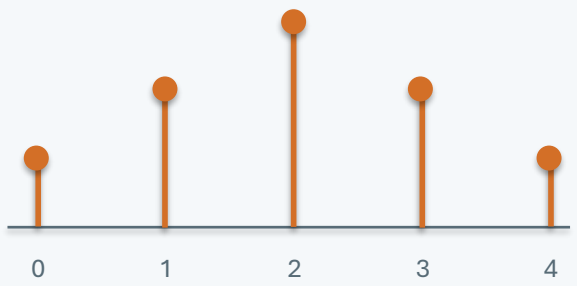
$h_1[n]$



$h_2[n]$



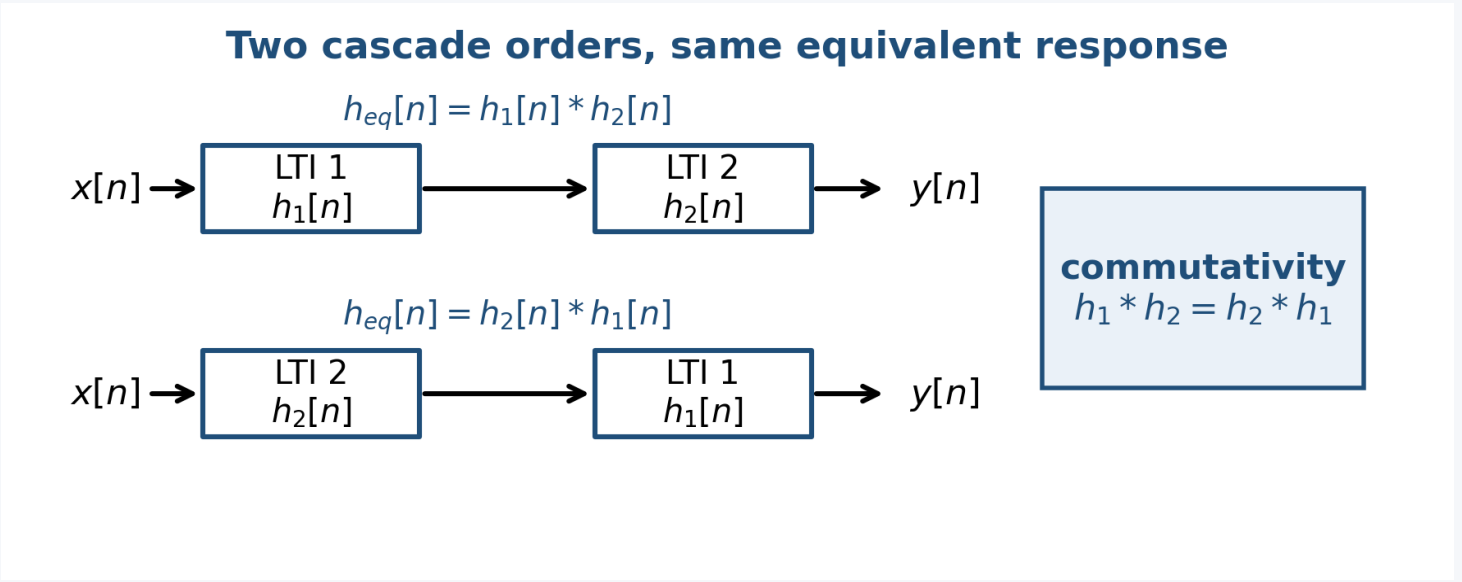
h_eq[n]



h_eq = [1, 2, 3, 2, 1]/9. The cascade is again a causal FIR, now with 5 taps.

Cascaded LTI systems — equivalent response

Why the order of two cascaded FIR systems does not matter



Convolution is commutative, so the cascade order can be interchanged.

Example 5 — cascaded FIR systems

Finding the equivalent impulse response of a cascade

Problem

Two causal FIR systems are connected in cascade with $h_1[n] = \{1,1,1,1\}$ and $h_2[n] = \{1,1,1\}$, both starting at $n = 0$. Find $h_{eq}[n]$.

Solution

1. Convolve the two moving-sum responses: $h_{eq}[n] = h_1[n] * h_2[n]$.
2. The overlap grows as 1, 2, 3, then stays at 3, and finally decreases as 2, 1.
3. So $h_{eq}[n] = \{1,2,3,3,2,1\}$.
4. The length is $4+3-1 = 6$, so the equivalent order is $M = 5$.

Final answer: $h_{eq}[n] = \{1,2,3,3,2,1\}$. The same result is obtained if the two systems are interchanged.

Key idea. Commutativity of convolution explains why the order of two cascaded LTI systems does not matter.

Cascade of two 3-point averages

Two stages create a triangular, five-tap equivalent impulse response.

First FIR

$h_1 = [1/3, 1/3, 1/3]$

Second FIR

$h_2 = [1/3, 1/3, 1/3]$

Equivalent FIR

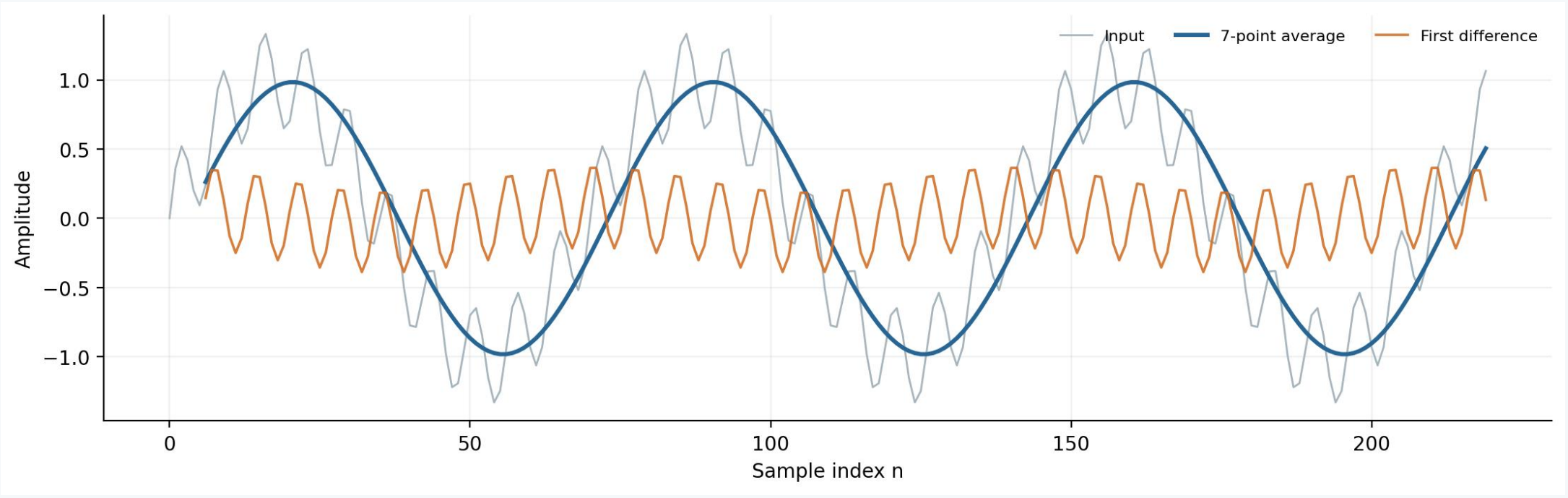
$h_eq = h_1 * h_2 = [1, 2, 3, 2, 1] / 9$

Interpretation

The effective weights are triangular, not uniform; reversing the order of two LTI stages does not change the final output.

A simple FIR listening / plotting demonstration

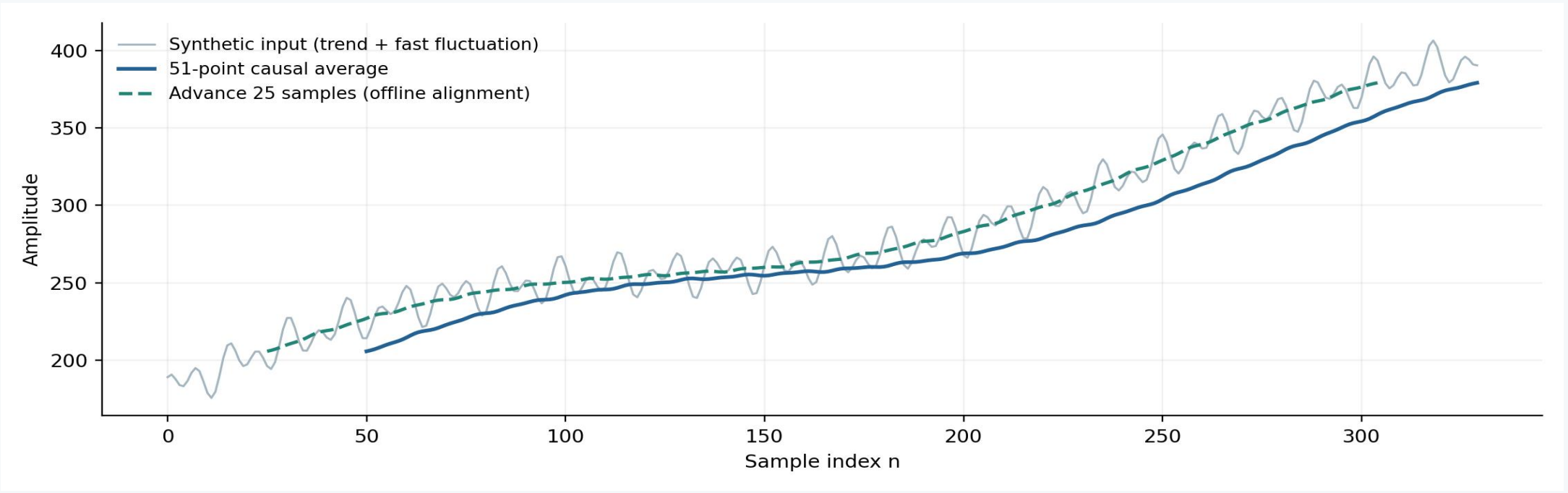
Compare a moving average and a first difference on a signal containing slow and fast variations.



Ask: Which output looks smoother? Which reacts most strongly to rapid changes? Why?

The chapter ends with a 51-point average

A causal 51-tap smoother creates a 25-sample group delay.



Dashed: same averaged result advanced 25 samples (offline visualization). It requires future data if used without delay.

Quick questions for the class

Check the differences between taps, delay, convolution and time invariance.

1. For b = [2, −1, 0, 3], what are the length and order?
2. What is h[n] for y[n] = x[n] − x[n−1]?
3. Can a causal FIR use x[n+1]? Why?
4. Why can a nonlinear system not be described by ordinary convolution with a fixed h[n]?

Notation and takeaways

Minimum language for FIR filters and convolution

x[n]	input sequence
y[n]	output sequence
b[k]	FIR coefficients
M	filter order
L	filter length = M+1

δ[n]	unit impulse
h[n]	impulse response
*****	convolution
LTI	linear and time invariant
causal	uses only present/past inputs

T5 — six things to remember

A compact checklist matching the final section of the written summary.

1 System

Map x[n] to y[n]

2 FIR

L=M+1 taps

3 Causality

No future samples

4 Impulse

h[k]=b[k]

5 Convolution

Sum shifted copies

6 LTI/cascade

$h_eq = h_1 * h_2$